

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5

Total Marks: 50

Annexure A — Code of Conduct

I **_PERAM VIVEKANANDA REDDY(192372330)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Vivek*

Annexure A — Technical Presentation (Question Paper)

1. Reactive Agent-Based Traffic Control System

Prepare a technical presentation on designing a Reactive Agent for a smart traffic control system. Explain how the agent responds to real-time traffic inputs, manages signal changes, and ensures minimal congestion using streaming data processing.

2. Goal-Based Agent for Healthcare Diagnosis

Develop a presentation on a Goal-Based Agent used in an AI healthcare system. Describe how the agent sets diagnostic goals, evaluates patient data, and selects optimal treatment pathways using decision-making logic and data pipelines.

3. Utility-Based Agent for E-Commerce Recommendations

Create a technical presentation on a Utility-Based Agent that generates personalized product recommendations. Explain how the agent assigns utility values to different options and optimizes user satisfaction using large-scale data processing.

4. Learning Agent for Personalized Education System

Present the design of a Learning Agent in an AI-driven educational platform. Explain how the agent improves performance over time using feedback, adapts content difficulty, and integrates with distributed data processing systems like PySpark.

5. Multi-Agent System for Disaster Management

Prepare a presentation on a Multi-Agent System where multiple agents (sensor agents, decision agents, communication agents) collaborate for disaster detection and response. Explain coordination strategies, data sharing, and real-time decision-making.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

TECHNICAL PRESENTATION CONTENT

The content below is organised as a slide-by-slide presentation script for each of the five topics — ready to be transferred into presentation software, with speaker explanations included under each slide.

1. Reactive Agent-Based Traffic Control System

SLIDE 1: Title & Problem Statement

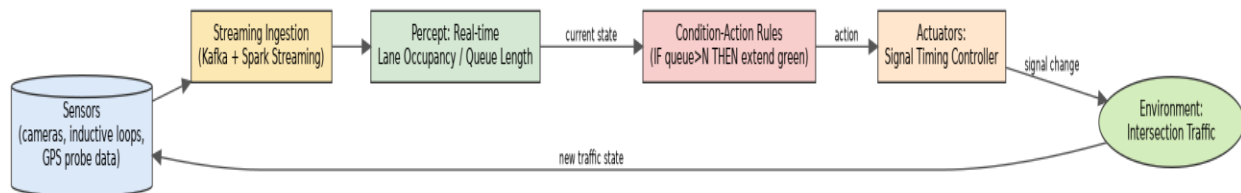
Smart Traffic Control Using a Reactive Agent — Minimising Congestion Through Real-Time, Rule-Based Response.

Problem: Fixed-timer traffic signals cannot adapt to real-time congestion, causing unnecessary queuing at low-traffic times and gridlock at high-traffic times. We need an agent that reacts instantly to the current traffic state without expensive lookahead planning, since intersection decisions must be made in milliseconds.

SLIDE 2: Why a Reactive Agent?

- A reactive agent selects actions purely from the current percept via condition-action rules — it has no internal model of the world and no memory of the past, making it extremely fast and predictable, ideal for a millisecond-latency control loop.
- Traffic-signal timing is a naturally reactive problem: the correct action (extend green / switch phase) depends almost entirely on the current queue length at each approach, not on a long-term plan.

SLIDE 3: Agent Architecture



Sensors (cameras, inductive loop detectors, GPS probe data from vehicles) continuously stream percepts into a Spark Streaming ingestion layer. Each micro-batch is converted into a real-time percept — current lane occupancy / queue length per approach — which is fed directly into the agent's condition-action rule set.

SLIDE 4: Condition-Action Rule Base

```
IF queue_length[NS] > 15 AND phase == 'EW_GREEN' AND elapsed_time > MIN_GREEN:
    ACTION = switch_phase('NS_GREEN')

IF queue_length[current_approach] > queue_length[opposing_approach] * 2:
    ACTION = extend_green(current_approach, +5_seconds)

IF pedestrian_button_pressed AND elapsed_time > MIN_GREEN:
    ACTION = schedule_pedestrian_phase()
```

Each rule maps directly from a percept condition to an actuator action with no intermediate reasoning step — this is the defining characteristic of a reactive (Simple Reflex) agent.

SLIDE 5: Streaming Data Processing Pipeline

```
from pyspark.streaming import StreamingContext
ssc = StreamingContext(sc, batchDuration=2) # 2-second reactive control window

sensor_stream = ssc.socketTextStream('sensor-gateway', 9999)
queue_lengths = sensor_stream.map(parse_sensor_reading) \
    .map(lambda r: (r.approach_id, r.vehicle_count)) \
    .reduceByKey(lambda a, b: a + b)

queue_lengths.foreachRDD(lambda rdd: rdd.foreach(apply_condition_action_rules))
```

A short 2-second micro-batch window keeps the control loop close to real time while still benefiting from Spark's distributed fault tolerance if a sensor feed briefly drops.

SLIDE 6: Ensuring Minimal Congestion

- Minimum/maximum green-time bounds prevent any single approach from starving others, even under reactive rule-firing.
- Neighbouring intersections' agents share their current phase state over a lightweight message bus, so a reactive 'green wave' can emerge along a corridor without any centralised planner.
- Because the agent holds no state beyond the current percept, it recovers instantly and safely after any sensor dropout or restart — an important robustness property for safety-critical infrastructure.

SLIDE 7: Conclusion

A reactive agent is well suited to traffic-signal control precisely because the correct response is almost always determined by the immediate sensory state; combined with a low-latency Spark Streaming pipeline, this achieves the millisecond-level responsiveness required to meaningfully reduce congestion, without the computational overhead of a full planning agent.

2. Goal-Based Agent for Healthcare Diagnosis

SLIDE 1: Title & Problem Statement

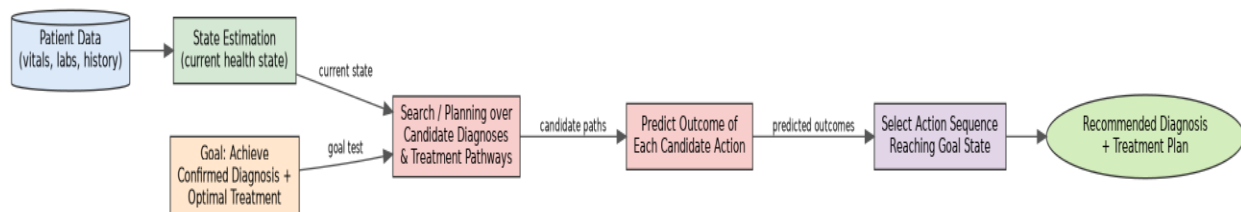
A Goal-Based AI Agent for Diagnostic Decision Support.

Problem: Diagnosis is not a single reflex response — it requires considering multiple candidate conditions, predicting how each candidate diagnosis would explain the observed symptoms, and choosing a path of tests/treatments that leads toward a clearly defined goal state (a confirmed diagnosis and an effective treatment), which a simple reactive agent cannot do.

SLIDE 2: Why a Goal-Based Agent?

- A goal-based agent maintains an explicit representation of the desired goal state and searches over possible action sequences to find one that achieves it — exactly matching how a clinician reasons: 'what sequence of tests/treatments gets this patient to a confirmed, treated state?'
- Unlike a reactive agent, it can look ahead: e.g., recognising that ordering test X now, even though not immediately conclusive, is the necessary path toward eventually confirming or ruling out a diagnosis.

SLIDE 3: Agent Architecture



Patient data (vitals, lab results, history) is first converted into a current state representation. The agent's explicit goal — 'reach a confirmed diagnosis with an optimal treatment plan' — is used as the goal test in a search over candidate diagnostic/treatment pathways. For each candidate path, the agent predicts the likely outcome (using a trained clinical model), and selects the path whose predicted outcome best satisfies the goal.

SLIDE 4: Decision-Making Logic

```
def goal_test(state):
    return state.diagnosis_confidence > 0.9 and state.treatment_plan is not None

def search_for_goal(initial_state, goal_test, actions):
    frontier = [initial_state]
    while frontier:
        state = frontier.pop(0)          # e.g. best-first by expected info gain
        if goal_test(state):
            return state.path            # sequence of tests/treatments taken
        for action in actions(state):    # e.g. order test, start treatment
            frontier.append(predict_next_state(state, action))
    return None  # no path found -> escalate to specialist
```

This is a textbook goal-directed search: candidate actions (ordering a test, starting a treatment) are expanded and evaluated against the explicit goal test until a satisfying path is found or the search is escalated to a human specialist.

SLIDE 5: Data Pipeline

Patient records stream from the hospital's EHR system into a distributed feature pipeline (similar in structure to the CO4 diabetes-prediction pipeline: cleaning, normalisation, and feature extraction) which continuously updates the 'current state' the agent reasons over. Historical outcome data (what happened to previous patients given similar states and actions) trains the outcome-prediction model that powers the agent's lookahead step.

SLIDE 6: Selecting the Optimal Treatment Pathway

- Among all action sequences that reach the goal test, the agent prefers the one with lowest predicted risk and cost (e.g., fewer invasive tests) — this can be layered with a utility function (see Topic 3) to break ties between multiple goal-satisfying paths, blending goal-based and utility-based reasoning as real clinical decision-support systems often do.
- A confidence threshold (e.g., $\text{diagnosis_confidence} > 0.9$) ensures the agent only commits to a goal-reaching path when sufficiently certain, otherwise it continues the search (ordering more tests) or defers to a human clinician.

SLIDE 7: Conclusion & Ethical Note

A goal-based agent is appropriate here because diagnosis genuinely requires multi-step reasoning toward an explicit goal, not an instantaneous reflex. As with any clinical AI system, the agent must operate as a decision-support tool with a human clinician retaining final authority, given the safety-critical consequences of diagnostic error.

3. Utility-Based Agent for E-Commerce Recommendations

SLIDE 1: Title & Problem Statement

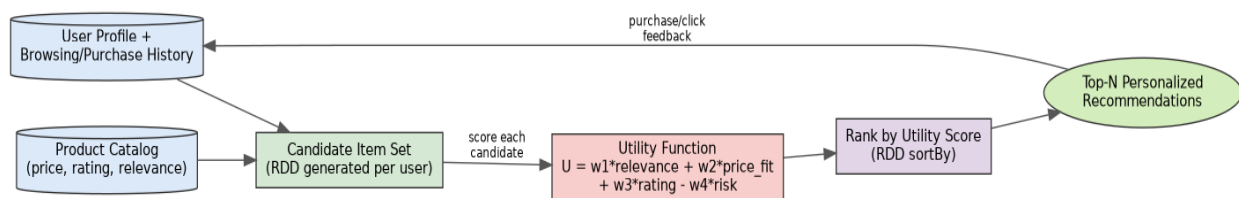
Maximising Customer Satisfaction with a Utility-Based Recommendation Agent.

Problem: Multiple candidate products may all 'satisfy the goal' of being relevant to a user, but they differ in price-fit, quality, and risk of return — a goal-based agent that only asks 'is this relevant?' cannot distinguish between them. We need an agent that can compare and rank many satisfactory options.

SLIDE 2: Why a Utility-Based Agent?

- A utility-based agent assigns a real-valued utility score to each possible outcome and chooses the action maximising expected utility — exactly what is needed to rank many 'good enough' product candidates against each other rather than just filtering for relevance.
- Utility naturally combines multiple, sometimes competing objectives (relevance, price-fit, rating, return-risk) into a single comparable number.

SLIDE 3: Agent Architecture



User profile and browsing/purchase history combine with the product catalog to generate a candidate item set per user (an RDD partitioned by user ID). The utility function scores every candidate; the agent ranks candidates by utility score and returns the top-N recommendations. The user's reaction (click, purchase, ignore) feeds back into their profile, continuously refining future utility calculations.

SLIDE 4: The Utility Function

```
def utility(user, item):
    relevance = cosine_similarity(user.profile_vector, item.feature_vector)
    price_fit = 1 - abs(item.price - user.typical_spend) / user.typical_spend
    rating_score = item.avg_rating / 5.0
    return_risk = item.historical_return_rate

    return (0.45 * relevance +
            0.20 * price_fit +
            0.25 * rating_score -
            0.10 * return_risk)

# Distributed scoring across all candidate (user, item) pairs
scored_rdd = candidates_rdd.map(lambda kv: (kv, utility(kv[0], kv[1])))
top_n_rdd = scored_rdd.sortBy(lambda kv: -kv[1]).take(N)
```

The weighted-sum utility function is deliberately simple and interpretable (important for explaining recommendations to stakeholders), while still capturing the multi-objective trade-off between relevance, affordability, quality, and risk.

SLIDE 5: Large-Scale Data Processing

- Candidate generation and utility scoring both run as RDD transformations partitioned by user ID, so the workload scales horizontally with the number of concurrent shoppers — a retailer with millions of active users can score all of them in parallel across the cluster.

- sortBy (a distributed sort/action) retrieves only the top-N candidates per user without ever materialising a full sorted list of the entire catalog, keeping the operation efficient at scale.

SLIDE 6: Optimizing User Satisfaction Over Time

- The utility function's weights (0.45, 0.20, 0.25, 0.10 above) are not fixed forever — they can be tuned per-user or globally via A/B testing and online learning based on which weighting best predicts actual satisfaction (purchase without return), turning the utility-based agent into a self-improving system over time.

SLIDE 7: Conclusion

A utility-based agent is the right model here because e-commerce recommendation is fundamentally a ranking problem among many acceptable options, not a binary relevance filter — and Spark's distributed RDD operations make scoring and ranking millions of user-item pairs computationally tractable in real time.

4. Learning Agent for Personalized Education System

SLIDE 1: Title & Problem Statement

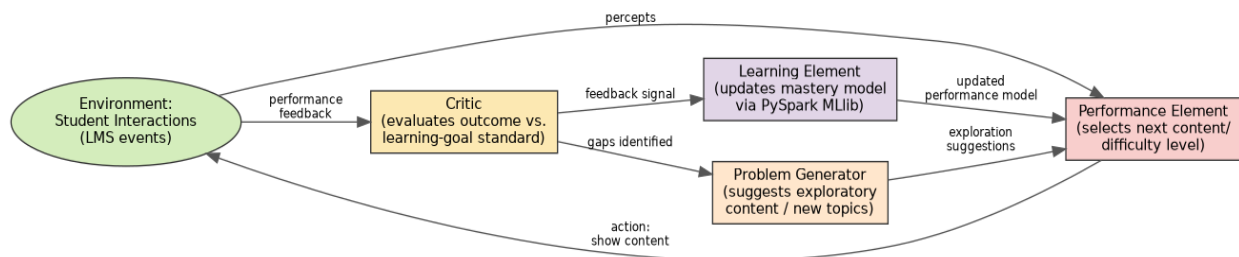
A Self-Improving Learning Agent for Adaptive, Personalized Education.

Problem: A fixed, hand-coded difficulty-adaptation rule (however well-designed today) will not stay optimal as new content is added and as the platform learns more about what actually helps students master material — the system needs to genuinely improve its own performance from experience, not just execute static rules or a one-time-trained model.

SLIDE 2: Why a Learning Agent?

- A learning agent is explicitly structured to improve its own decision-making over time from feedback, using the classical four-component architecture: Performance Element, Critic, Learning Element, and Problem Generator.
- This matches the requirement directly: the platform should 'improve performance over time using feedback' and 'adapt content difficulty' — both are definitional properties of a learning agent, not a simple reflex or one-shot goal/utility agent.

SLIDE 3: Four-Component Agent Architecture



Component	Role in this System
Performance Element	Selects the next content item / difficulty level to show the student, given their current mastery-model state (this is the 'acting' part of the agent).
Critic	Observes the outcome (quiz score, time-on-task, completion) and evaluates it against a fixed performance standard (e.g., expected mastery gain), producing a feedback signal — independent of the Performance Element so evaluation isn't biased by the agent's own choices.
Learning Element	Uses the Critic's feedback signal to update the underlying mastery-prediction model (e.g., retraining a distributed model via PySpark MLlib on accumulated interaction data).
Problem Generator	Deliberately suggests exploratory content (slightly outside the model's current confidence) so the agent keeps gathering informative data rather than only ever reinforcing its current beliefs — analogous to controlled exploration in reinforcement learning.

SLIDE 4: PySpark Integration

```
from pyspark.mllib.tree import GradientBoostedTrees

# Learning Element: periodic distributed retraining on accumulated interaction RDD
interaction_rdd = sc.textFile('lms/interactions/*.json').map(parse_interaction)
training_rdd = interaction_rdd.map(build_labeled_point) # (mastery_gain, features)

updated_model = GradientBoostedTrees.trainRegressor(
    training_rdd, categoricalFeaturesInfo={}, numIterations=50)
```

```
# Performance Element: score candidate next-content items using the current model
def select_next_content(student_state, candidate_items, model):
    scored = [(item, model.predict(features(student_state, item))) for item in candidate_items]
    return max(scored, key=lambda x: x[1])[0]
```

SLIDE 5: Adapting Content Difficulty

- The Performance Element continuously compares predicted mastery-gain across candidate next-items at several difficulty levels, choosing the one the current model predicts will most improve the student's mastery without excessive frustration (a mid-range predicted-success-probability heuristic, similar to the 'zone of proximal development' teaching principle).
- Because the Learning Element retraines on the growing pool of (state, action, outcome) data from all students, difficulty-adaptation genuinely improves platform-wide over time, not just for one individual student's session.

SLIDE 6: Scalability

Interaction logging, feature construction, and model retraining are all implemented as RDD-based distributed operations, so the pipeline scales to 'thousands of learners' by adding Spark executors, exactly as required by the assignment brief; model retraining can run as a scheduled distributed batch job (e.g., nightly) without disrupting the low-latency, model-serving Performance Element used during live sessions.

SLIDE 7: Conclusion

Framing this system explicitly as a four-component learning agent (rather than a single static model) makes clear how the platform keeps improving after deployment — the Critic and Learning Element together close a genuine feedback loop that a simple reflex or one-shot utility agent does not have.

5. Multi-Agent System for Disaster Management

SLIDE 1: Title & Problem Statement

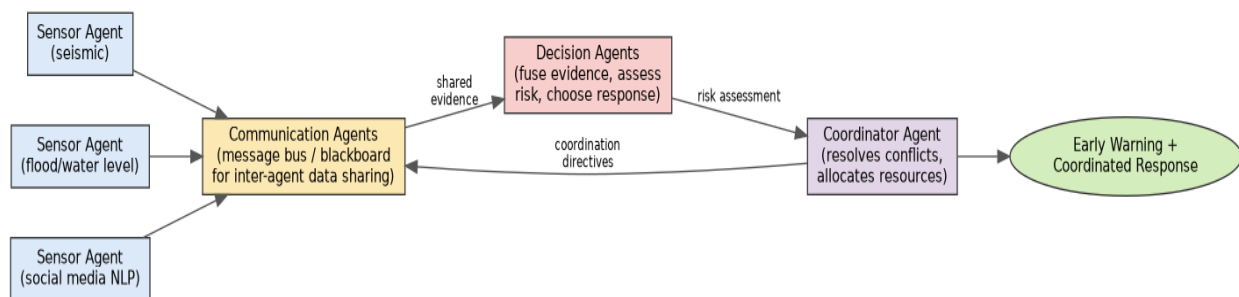
A Coordinated Multi-Agent System (MAS) for Disaster Detection and Response.

Problem: Disaster detection and response naturally involves distinct, geographically and functionally distributed responsibilities (sensing, deciding, communicating) that cannot be handled by a single monolithic agent without creating a fragile central bottleneck — a multi-agent architecture is needed so specialised agents can act autonomously while still coordinating toward a shared goal.

SLIDE 2: Why a Multi-Agent System?

- Different sub-tasks require fundamentally different competencies (raw sensor interpretation vs. risk-assessment reasoning vs. inter-agency communication) — a MAS lets each agent type specialise, be developed/scaled independently, and continue operating even if another agent type is temporarily degraded.
- Disaster response is inherently distributed geographically, so agents naturally map onto regions/sensor-sites rather than a single global controller.

SLIDE 3: Multi-Agent Architecture



Sensor Agents (one per data modality — seismic, flood/water-level, social-media NLP) each independently monitor their feed and publish structured observations. Communication Agents implement a shared message bus / blackboard so any agent can publish evidence and any other agent can subscribe to relevant updates, decoupling producers from consumers. Decision Agents fuse evidence across modalities and regions to assess risk and propose responses. A Coordinator Agent resolves conflicting proposals (e.g., two decision agents requesting the same limited rescue resources) and issues final coordination directives.

SLIDE 4: Coordination Strategies

- Blackboard architecture: agents communicate indirectly via a shared, structured data store (the 'communication agent' layer) rather than direct point-to-point messaging, which scales far better as the number of agents grows and allows new agent types to be added without modifying existing agents.
- Contract-net protocol: when a Decision Agent needs a specific resource (e.g., a drone survey of a region), it broadcasts a task announcement; available resource-owning agents bid, and the Coordinator Agent awards the task to the best bid — a well-established MAS coordination pattern for decentralised task allocation.
- Conflict resolution by priority + negotiation: when two Decision Agents' recommended responses compete for the same limited resource, the Coordinator Agent applies a priority rule (e.g., higher estimated casualties-at-risk wins) with a negotiation fallback for near-ties.

SLIDE 5: Data Sharing & Real-Time Decision-Making

```
# Simplified blackboard-style shared-state update (conceptually backed by a
# distributed key-value/message store such as Kafka + a Spark Streaming consumer)

def sensor_agent_publish(region, modality, reading, blackboard):
    blackboard.update(key=(region, modality), value=reading, ts=now())

def decision_agent_evaluate(region, blackboard):
    evidence = blackboard.get_all(region)      # reads across all modalities
    risk_score = fuse_evidence(evidence)       # weighted fusion, as in Task 2 (AT1)
    if risk_score > THRESHOLD:
        return propose_response(region, risk_score)
    return None

def coordinator_agent_resolve(proposals):
    return sorted(proposals, key=lambda p: -p.priority) # allocate resources by priority
```

SLIDE 6: Real-World Applicability

- This MAS pattern mirrors real emergency-management doctrine (distinct sensing, assessment, and coordination roles across agencies), making the design directly interpretable and trustable by human emergency responders who will work alongside it.
- Because agents communicate via a shared blackboard rather than tight point-to-point coupling, the system degrades gracefully — losing one Sensor Agent (e.g., a damaged seismic station) reduces evidence quality for that region but does not crash the overall system.

SLIDE 7: Conclusion

Modelling disaster management as a coordinated multi-agent system — rather than one large monolithic model — mirrors the genuinely distributed, multi-competency nature of real disaster response, and provides natural fault-tolerance and extensibility as new sensor types or response agencies are added.